In [ ]:
```
Name : Omkar Hulawale
Class , Batch : TE-A , A3 Batch
Roll No: 13165
```

In [49]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, reca
```

In [51]:
```
df=pd.read_csv(r'C:\Users\CNLAB07\Downloads\iris.csv')
df
```

Out[51]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa
...	...	...	...	...	...	...
145	146	6.7	3.0	5.2	2.3	Iris-virginica
146	147	6.3	2.5	5.0	1.9	Iris-virginica
147	148	6.5	3.0	5.2	2.0	Iris-virginica
148	149	6.2	3.4	5.4	2.3	Iris-virginica
149	150	5.9	3.0	5.1	1.8	Iris-virginica

150 rows × 6 columns

In [53]:
```
# Display the first few rows
print(df.head())
```

```
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

In [55]:
```
# Encode target labels if necessary
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df['Species'] = label_encoder.fit_transform(df['Species'])
```

In [57]:
```
# Split into features (X) and target (y)
X = df.drop(columns=['Species'])
y = df['Species']
```

In [59]:
```
# Load Iris dataset
X = df.iloc[:, :-1].values  # Features
y = df.iloc[:, -1].values   # Target
```

In [61]:
```
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

In [63]:
```
# Train Naïve Bayes classifier
model = GaussianNB()
model.fit(X_train, y_train)
```

Out[63]:
```
▾ GaussianNB

GaussianNB()
```

In [65]:
```
# Predictions
y_pred = model.predict(X_test)
```

In [67]:
```
# Compute Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
tp = cm[0, 0]  # True Positives
tn = cm[1, 1]  # True Negatives
fp = cm[1, 0]  # False Positives
fn = cm[0, 1]  # False Negatives
```

In [69]:
```
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()[:4]  # Extract values
```

In [71]:
```
# Performance metrics
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')
```

In [73]:
```
# Print results
print(f"Confusion Matrix:\n{cm}")
print(f"True Positives (TP): {TP}")
print(f"False Positives (FP): {FP}")
print(f"True Negatives (TN): {TN}")
print(f"False Negatives (FN): {FN}")
print(f"Accuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```

```
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
True Positives (TP): 0
False Positives (FP): 0
True Negatives (TN): 10
False Negatives (FN): 0
Accuracy: 1.00
Error Rate: 0.00
Precision: 1.00
Recall: 1.00
F1 Score: 1.00
```

In [75]:
```
# Visualizing Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoder.classe
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```